Scope

The work in this script focuses on:

- Creating the MySQL database schema
- Building the four database tables:
    - App
    - Country
    - Raw_Reviews
    - Cleaned_Reviews

- Importing cleaned datasets into MySQL
- Testing repeated ingestion using the same RSS database
- Testing ingestion using a new RSS database
- Identifying and fixing ingestion design issues
- Documenting validation results

Original Design/Issues and Solution

- Use Python-generated ‘raw_review_pk’ based on the RSS row order as a dedup key.

    - The same review may receive different primary keys across ingestion runs when the RSS order changes with the new RSS data.
    - No stable identifier exists for incremental loading.
    - Relationships between Raw_Reviews and Cleaned_Reviews become unreliable if primary keys change.

Solution:

Construct a more stable dedup key：

- dedup_key = app_id + "|" + country_name + "|" + review_id

We selected ‘app_id’, ‘country_name’, and ‘review_id’ because, together, they uniquely identify a review within its business context. This composite key mitigates the risk of duplicate identification caused by repeated RSS extraction or by the potential reuse of the same review_id across different applications or App Store regions.

How Incremental Loading Works

Step 1: Fetch Review Data

Retrieve review records from the Apple RSS feed for each selected application and App Store country.

Step 2: Generate a Stable Business Key

- Before inserting any record into the database, generate a stable business identifier (dedup_key) for each review.
- dedup_key = app_id + "|" + country_id + "|" + review_id
- This composite key uniquely identifies a review within its business context and remains stable across repeated ingestion runs, regardless of changes in the RSS ordering.

Step 3: Load Existing Business Keys

- Retrieve all existing dedup_key values from the Raw_Reviews table and store them in a Python set (existing_keys).

Step 4: Perform Duplicate Detection

```
inserted = 0
duplicates = 0
errors = 0

for row in raw_reviews_mysql.itertuples(index=False):
    if row.dedup_key in existing_keys:
        duplicates += 1
        continue

    try:
        cursor.execute(sql, tuple(row))
        inserted += 1
        existing_keys.add(row.dedup_key)
    except mysql.connector.IntegrityError:
        duplicates += 1
    except Exception as e:
        print(f"Error: {e}")
        errors += 1

conn.commit()
```

For each newly collected review:

- If the ‘dedup_key’ already exists in existing_keys
  - Skip the record (no insertion).
- Otherwise
  - Insert the review into the ‘Raw_Reviews’ table.
  - Immediately add the ‘new dedup_key’ to existing_keys to prevent duplicate insertion within the same ingestion run.

This ensures that only previously unseen reviews are inserted into the database.

Step 5: Generate the Database Primary Key

- After a successful insertion, MySQL automatically generates a unique ‘raw_review_pk’ using the AUTO_INCREMENT mechanism.
- Once a review has been inserted, its **assigned raw_review_pk remains unchanged**. New reviews receive new primary keys through the AUTO_INCREMENT mechanism, while previously inserted reviews retain their original primary keys.

Step 6: Retrieve the Database Primary Key back to the Python Raw_reviews table, and make it the Foreign Key for the Cleaned_reviews table.

```
mapping_sql = """
SELECT raw_review_pk, dedup_key, record_remove_reason
FROM raw_reviews
"""

cursor.execute(mapping_sql)
raw_mapping = pd.DataFrame(
    cursor.fetchall(),
    columns=["raw_review_pk", "dedup_key", "record_remove_reason"]
)
```

```
# ============================================================
# MODIFICATION 4: Use INNER JOIN to filter out invalid records
# ============================================================
cleaned_reviews_mysql = cleaned_reviews_mysql.merge(
    raw_mapping_valid,
    on="dedup_key",
    how="inner"
)

cleaned_reviews_mysql = cleaned_reviews_mysql.dropna(subset=['raw_review_pk'])

print(f"Cleaned records ready for insertion: {len(cleaned_reviews_mysql)}")

cleaned_reviews_mysql = cleaned_reviews_mysql[
    ["raw_review_pk", "content_cleaned", "date_transformed"]
]
```

- Retrieve the generated ‘raw_review_pk’, ‘dedup_key’, and ‘record_remove_reason’ from the Raw_Reviews table.
- Keep only records with a NULL record_remove_reason to create the mapping between valid raw reviews and their database primary keys.
- Perform an INNER JOIN between the cleaned review dataset and the valid raw review mapping using dedup_key.

Step 7: Insert the Cleaned_reviews table into the Database

Limitation

- The current design uses the composite dedup_key to reliably identify duplicate records for incremental loading. Once a duplicate record is detected, it is skipped before being inserted into the database.
- As a result, only the first occurrence of each dedup_key is retained in the Raw_Reviews table, while subsequent duplicate records are discarded. Consequently, duplicate source records cannot be traced or analysed after the ingestion process has completed.

For data extraction on 7/19/2026（09:00-）

- **Ingestion run one (first ingestion)**

Table	Records Inserted	Duplicates	Errors
App	5	0	0
Country	5	0	0
Raw_Reviews	10784	0	0
Cleaned_Reviews	8853	0	0

- **Ingestion run two (first ingestion)**

Table	Records Inserted	Duplicates	Errors
App	0	5	0
Country	0	5	0
Raw_Reviews	0	10784	0
Cleaned_Reviews	0	8792 (fewer than expected)	0

-The second ingestion run correctly identified all existing records and skipped duplicate insertions

Limitation

The skipped duplicates seem a little bit weird here, since it skips fewer records than expected:

- Ingestion run one inserts 8853 records into the database;
- Ingestion run two with the same RSS should skip the same record number as 8853, while it virtually shows 8792 records are skipped, which is fewer than our expectation.

However, we think this limitation does not impact our final data results:

- Since we run the same RSS data for the first two runs, the insertion should be 0.
- Our Ingestion run 2 shows insertion is constantly 0, showing that the skipping logic works successfully for the ingestion runs using the same data.
- This difference is likely caused by slight inconsistencies in rerunning language detection across runs.

Quality Flags (records needed to be removed)

Sum(condition=True)

Duplicate Records	Missing Review Content	Non-English Review Content	Empty Review After Cleaning
0	0	1931	0

Missing Review Content

-Reviews with empty or placeholder values (e.g., "", "NA", "N/A", "Unknown") are flagged because they do not contain meaningful textual information.

Duplicate Records

-Duplicate reviews are identified using the “review_id”. Only the first occurrence is retained, while subsequent duplicates are flagged to avoid repeated records in downstream analysis.

Non-English Review Content

-Since the project focuses on English-language text analysis, reviews whose detected language is not English are flagged for removal.

Empty Review After Cleaning

-After text preprocessing (such as removing URLs, emojis, punctuation, and other non-textual characters), some reviews may become empty. These records are also flagged because they no longer contain analyzable content.

The quality flags were designed to document why a raw review should be excluded from the **Cleaned_Reviews dataset** while preserving the original record for traceability.

Instead of deleting records immediately, each raw review is evaluated against a series of predefined quality checks. If a record fails a quality check, a removal reason is recorded in the ‘record_remove_reason’ field. This allows every removed record to remain traceable in the **Raw_Reviews table**.

1931 raw review records did not have corresponding records in Cleaned_Reviews because they were intentionally excluded during the cleaning stage due to non-English content.

- **Table relationship(Across the same RSS ingestion runs)**

  1. All foreign key relationships remained valid after the ingestion process. No orphan records were detected in any of the three relationships.

  2. The database query results are consistent with the Python ETL Statistics:

For data extraction on 7/19/2026 (9:59)

- **Ingestion run one**

Table	Records Inserted	Duplicates	Errors
App	0	5	0
Country	0	5	0

Raw_Reviews	147	10784	0
Cleaned_Reviews	115	8779	0

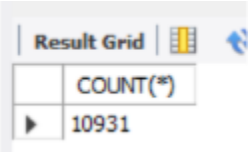
Quality Flags

Sum(condition=True)

Duplicate Records	Missing Review Content	Non-English Review Content	Empty Review After Cleaning
0	0	1963	0

Test Result Validation
(Validation of the Ingestion Run Using New RSS Data Against Previous Runs)

SELECT COUNT(*)
FROM Raw_Reviews;

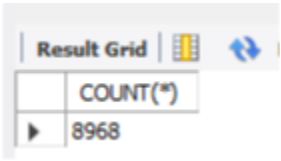


Current Raw_Reviews count - Raw_Reviews count in ingestion run 1:

- 10931-10784=147

The increase of 147 records matches the number of newly inserted records reported during the ingestion of the new RSS dataset. This confirms that all newly identified raw reviews were successfully inserted into the ‘Raw_Reviews’ table.

SELECT COUNT(*)
FROM Cleaned_Reviews;

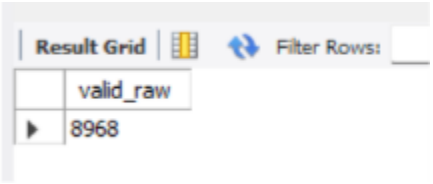


Current Cleaned_Reviews count - Cleaned_Reviews count in ingestion run 1:

- 8968-8853=115

The increase of 115 records matches the number of newly inserted records reported for the Cleaned_Reviews table, confirming that the cleaned review insertion process completed successfully.

SELECT COUNT(*) AS valid_raw
FROM Raw_Reviews
WHERE record_remove_reason IS NULL;

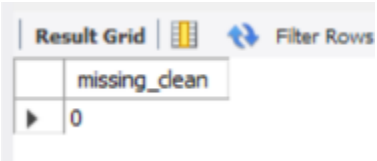


Valid_raw count =Cleaned_reviews count in the current run

- 8968=8968

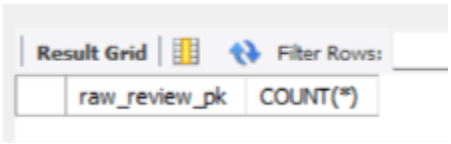
The number of valid records in ‘Raw_Reviews’ exactly matches the total number of records in ‘Cleaned_Reviews’ (8968). This confirms that every record in the ‘Cleaned_Reviews’ table passed through the quality check filtering.

```
SELECT COUNT(*) AS missing_clean
FROM Raw_Reviews r
LEFT JOIN Cleaned_Reviews c
ON r.raw_review_pk = c.raw_review_pk
WHERE r.record_remove_reason IS NULL
  AND c.raw_review_pk IS NULL;
```



The query returned **0** missing records, indicating that every valid raw review has a corresponding record in ‘Cleaned_Reviews’. Together with the previous count validation, this confirms that the filtering and insertion process was completed without missing any eligible records.

```
SELECT raw_review_pk, COUNT(*)
FROM Cleaned_Reviews
GROUP BY raw_review_pk
HAVING COUNT(*) > 1;
```



The query returned no duplicate ‘raw_review_pk’ values in the ‘Cleaned_Reviews’ table, confirming that each raw review is associated with at most one cleaned review. This is consistent with the intended one-to-zero-or-one relationship between the two tables.

The current Ingestion run 1 with new RSS data yielded Non-English Review Content count(Total record_remove_reason count) - (Total record_remove_reason count) for the first ingestion run = 1963- 1931 = 32

Which is aligned with the current run insertion difference between the two tables
147-115=32

- Ingestion run with the new RSS data produced: **32** additional records have the record_removal_reason compared with the first ingestion run;

- This matches the difference in the number of inserted records between the ‘Raw_Reviews’ and ‘Cleaned_Reviews’ tables;

- This indicates that the insertion difference between the two tables is expected, as these 32 records were intentionally excluded from the ‘Cleaned_Reviews’ table due to the Non-English Review Content quality filter.

Therefore, the observed difference reflects the data cleaning rules rather than an ingestion or database inconsistency.

Conclusion

Based on the validation results above, we conclude that the dynamic ingestion pipeline performs as intended. While a minor limitation was observed in the reported duplicate-skipping count, the validation results confirm that no duplicate records were inserted, no eligible records were lost during the ingestion process, and the expected filtering rules were correctly applied. Therefore, the resulting dataset is considered reliable and suitable for subsequent analysis.